

Deep Learning Algorithms and Implementations Report

Redo experiments from Liu et al. (2023) that exploit sparsity to accelerate LLM inference

Group 5: Shao-Chun Ho, Yu-Chia Chang

Date: 12/14/2025

1 Introduction

Large Language Models (LLMs) have achieved amazing results in many tasks. However, they are very large and require a lot of computing power, making them slow and expensive to run. To solve this problem, recent research proposed Contextual Sparsity [1]. The key idea is that for any specific input, the model does not need to use all its parameters. Instead, it can dynamically select and activate only a small subset of parameters without hurting performance.

In this project, we aim to verify and reproduce these findings. We investigate how contextual sparsity works across different model architectures and sizes. Our experiments cover a wide range of setups: we start with GPT-2 variants (Small and XL) on language modeling tasks (CBT-CN, LAMBADA and WikiText2), and evaluate the modern Llama3-8B model on math reasoning tasks (MGSM). **We specifically focus on two main questions: how the "complexity" of a task affects sparsity, and how different parts of the model (Attention vs. FFN) react when we reduce the number of activated parameters.**

2 Background

2.1 Transformer Architecture

The transformer architecture is widely used in modern deep neural networks. Its self-attention mechanism enables the model to capture relationships between tokens effectively. A transformer block consists of two main components: an attention module and a feed-forward network module. Large models stack many of such blocks to increase capacity. In other words, the architecture of a transformer-based neural network can be viewed as an interleaved sequence of attention modules and feed-forward network modules.

2.1.1 Attention Module

The attention module determines how much each token in a sequence should focus on every other token. Its core operation is **scaled dot-product attention (SDPA)**, which captures the semantic relationships between the current token and previous tokens. Specifically, given an input token x and a sequence of tokens X (including x itself), each token is first projected into query, key, and value vectors by the three weight matrices:

$$q = xW^Q, \quad K = XW^K, \quad V = XW^V$$

The scaled dot-product attention (SDPA) is then computed as:

$$SDPA(q, K, V) = softmax\left(\frac{qK^T}{\sqrt{d_k}}\right)V$$

In the transformer architecture, attention is implemented as **multi-head attention (MHA)**. Here, the query, key, and value vectors are split into multiple “heads,” with each head computing scaled dot-product attention independently. The outputs of all heads are then concatenated and projected to form the final output. Formally, for N_h attention heads, the output of the attention module can be computed as:

$$q = xW^Q, \quad K = XW^K, \quad V = XW^V$$

We split the query, key, value vectors into multiple heads:

$$q = [q_1 \ q_2 \ \dots \ q_{N_h}] \tag{1}$$

$$K = [K_1 \ K_2 \ \dots \ K_{N_h}] \tag{2}$$

$$V = [V_1 \ V_2 \ \dots \ V_{N_h}] \tag{3}$$

$$\tag{4}$$

Next, we perform scaled dot-product attention on each head:

$$Head_i = SDPA(q_i, K_i, V_i) = softmax\left(\frac{q_i K_i^T}{\sqrt{d_k}}\right)V_i, \quad i = 1, 2, 4, \dots, N_h$$

Finally, the outputs of all heads are concatenated and projected to produce the final output of the attention module:

$$\text{output} = [Head_1 \ Head_2 \ \dots \ Head_{N_h}] W^O$$

2.1.2 Feed-Forward Network Module

On the other hand, the feed-forward network (FFN) module consists of two linear projection layers (up-projection and down-projection), separated by a nonlinear activation function. In particular, the design of the up-projection layers has evolved over successive transformer architectures.

In the original transformer design (adopted by models such as GPT-2 and BERT), the FFN can be expressed as:

$$FFN(x) = \sigma(xW^U + b^U)W^D + b^D,$$

where W^U, b^U denote the weight matrix and bias of the up-projection layer, σ represents the activation function, and W^D, b^D correspond to the weight matrix and bias of the down-projection layer.

Recent transformer architectures, such as LLaMA and Qwen, adopt a modified formulation called Gated Linear Unit (GLU) that introduces an additional weight matrix W^G in the up-projection layer:

$$FFN(x) = (\sigma(xW^U) \odot (xW^G))W^D,$$

where $\odot$ denotes element-wise multiplication, and note that the bias terms are often ignored in such designs. This design replaces the single up-projection layer with two parallel up-projection layers. One is passed through an activation function (like Swish), and the other acts as a "gate" that controls what information passes through. This gating mechanism has been shown to improve model quality significantly.

2.2 Contextual Sparsity

Recent research [1] reveals that transformer-based LLMs exhibit contextual sparsity, a phenomenon where only a fraction of the total model parameters are significantly engaged for any given input. Rather than uniformly utilizing the entire network, the model selectively activates distinct subsets of model parameters depending on the semantic or syntactic characteristics of the sequence. It is also shown that this property exists for both the attention module and the feed-forward network module, where specific tokens may rely only a small subset of attention heads or a small subset of columns/rows of parameters of the FFN. It suggests that transformers implicitly learn to route information through specialized subspaces of the parameter space. Leveraging this contextual sparsity allows for reduced computation and memory overhead by dynamically bypassing non-essential parameters, thereby improving efficiency without compromising model quality.

3 Methodology

In this study, we shift the evaluation focus from sparsity to **density ratios**. We define the density ratio, r , as the fraction of model parameters actively engaged during the computation of the multi-head attention (MHA) or feed-forward network (FFN) modules. Below, we formalize the mechanism for selective parameter activation governed by this ratio.

3.1 Additive Decomposition of Modules

We first establish that both the MHA and FFN modules exhibit an additive structure. This property allows them to be decomposed into M independent sub-networks, whose outputs aggregate to form the total module output. Let $F(x, \theta)$ denote the full module computation on

input x with parameter set θ . We define $f(x, \theta_i)$ as the computation of a single independent sub-network i . This relationship is formulated as:

$$F(x, \theta) = \sum_{i=1}^M f(x, \theta_i) \quad (5)$$

Here, the parameter sets $\{\theta_i\}_{i=1}^M$ form a disjoint partition of θ where each subset is of equal size, satisfying $\bigcup_{i=1}^M \theta_i = \theta$ and $\theta_i \cap \theta_j = \emptyset$ for all $i \neq j$.

3.1.1 Decomposition of Multi-Head Attention

The computation within the MHA module decomposes into the **summation of outputs from distinct attention heads**. Given a query token x , context tokens X , and the set of projection parameters $\{W^Q, W^K, W^V, W^O\}$, we partition the parameters across N_h heads. Let d_{model} be the model dimension and $d_k = d_{model}/N_h$ be the head dimension. The projection matrices for the i -th head are defined by partitioning the full weights: $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{model} \times d_k}$ correspond to the column-wise splits of the input projections, while $W_i^O \in \mathbb{R}^{d_k \times d_{model}}$ corresponds to the row-wise split of the output projection. By decomposing the output projection matrix W^O into these row blocks W_i^O , the operation can be expressed as:

$$\text{MHA}(\{x, X\}, \{W^Q, W^K, W^V, W^O\}) = [\text{Head}_1 \quad \text{Head}_2 \quad \dots \quad \text{Head}_{N_h}] W^O \quad (6)$$

$$= \sum_{i=1}^{N_h} \text{Head}_i W_i^O \quad (7)$$

$$= \sum_{i=1}^{N_h} \left(\text{softmax} \left(\frac{x W_i^Q (X W_i^K)^T}{\sqrt{d_k}} \right) X W_i^V \right) W_i^O \quad (8)$$

$$= \sum_{i=1}^{N_h} \text{Attention}(\{x, X\}, \{W_i^Q, W_i^K, W_i^V, W_i^O\}) \quad (9)$$

3.1.2 Decomposition of Feed-Forward Network

Similarly, the FFN module can be expressed as a **summation of outputs from each intermediate feature dimension**. Given an input token x and parameters $\{W^U, b^U, W^D, b^D\}$, we partition the parameters into N_f intermediate features (neurons). The operation can be expressed as:

$$\text{FFN}(x, \{W^U, b^U, W^D, b^D\}) = \sigma(x W^U + b^U) W^D + b^D \quad (10)$$

$$= \sum_{i=1}^{N_f} \left(\sigma(x W_i^U + b_i^U) W_i^D + \frac{b^D}{N_f} \right) \quad (11)$$

$$= \sum_{i=1}^{N_f} \text{FFN}(x, \{W_i^U, W_i^D, b_i^U, \frac{b^D}{N_f}\}) \quad (12)$$

where W_i^U is the i -th column vector of the up-projection matrix W^U (mapping the input to a single feature scalar), and W_i^D is the i -th row vector of the down-projection matrix W^D (mapping that scalar back to the output dimension). Note that the output bias b^D is distributed across the summation. Additionally, this additive property extends naturally to the Gated Linear Unit (GLU) architecture:

$$\text{FFN}_{\text{GLU}}(x, \{W^U, W^G, W^D\}) = (\sigma(xW^U) \odot (xW^G))W^D \quad (13)$$

$$= \sum_{i=1}^{N_f} (\sigma(xW_i^U) \odot (xW_i^G))W_i^D \quad (14)$$

$$= \sum_{i=1}^{N_f} \text{FFN}_{\text{GLU}}(x, \{W_i^U, W_i^G, W_i^D\}) \quad (15)$$

3.2 Magnitude-Based Activation Selection

Given a target density ratio r , we approximate the full computation using only $K = \lfloor r \times M \rfloor$ sub-networks. To minimize the approximation error, we selectively activate the sub-networks that contribute most significantly to the final aggregation. Adopting the method from [1], we utilize the output norm as a proxy for contribution magnitude. Formally, we determine the activation index set $\mathcal{S}$ by selecting the indices corresponding to the largest output norms:

$$\mathcal{S} = \arg \text{top-}K (\|f(x, \theta_i)\|_2)_{i \in \{1, \dots, M\}}$$

Consequently, the approximated output is given by

$$F_{\text{sparse}}(x, \theta) = \sum_{i \in \mathcal{S}} f(x, \theta_i).$$

4 Experiment Results

Here, we evaluate the activation sparsity patterns of varying Transformer architectures under different conditions. We aim to understand how task complexity, model scaling, and specific module types (Attention vs. FFN) influence the intrinsic sparsity of large language models.

4.1 Impact of Task Complexity on Activation Sparsity

4.1.1 Model: GPT2-XL, Target Module: FFN, Dataset: [CBT-CN, LAMBADA]

We first distinguish the two benchmarks utilized based on their complexity. The CBT-CN (Children’s Book Test - Common Nouns) dataset is derived from narrative texts in children’s books and evaluates the model’s ability to predict a missing common noun using primarily local context. In contrast, the LAMBADA dataset is specifically designed to test the understanding of broad discourse context. It requires the model to predict the final word of a passage, which is difficult to solve using only the immediate sentence but solvable when considering the full

paragraph, thereby necessitating the tracking of long-range dependencies. Furthermore, CBT-CN provides a predefined set of candidate words from which the model selects the answer, whereas LAMBADA imposes no such constraint. This lack of candidate restriction further increases the difficulty of LAMBADA compared to CBT-CN.

Figure 1 illustrates the performance of the GPT2-XL model on these differing tasks. The simpler CBT-CN task shows high resilience to sparsity, maintaining near-peak accuracy (around 90%) even as the density ratio drops to 0.4. A sharp decline in accuracy is only observed when the density ratio falls below 0.3. Conversely, the more complex LAMBADA dataset proves much more sensitive. Accuracy begins to degrade once the density ratio drops below 0.7 and continues to decline steadily thereafter. By a density ratio of 0.5, LAMBADA accuracy has already decreased markedly relative to the dense model, while CBT-CN performance remains largely unaffected.

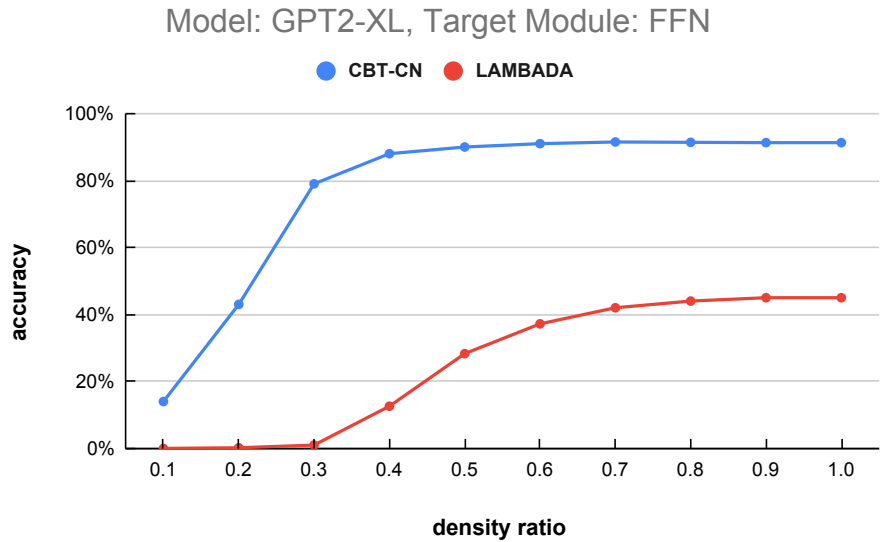


Figure 1

4.1.2 Key Findings

The results indicate that task complexity significantly influences a model’s tolerance for sparsity. Simpler tasks like CBT-CN allow for aggressive sparsity of FFN parameters (up to 60-70%) with minimal performance loss. However, more complex tasks like LAMBADA require a much higher density of active parameters to maintain performance, suggesting that harder problems engage a larger portion of the model’s capacity.

4.2 Scalability Analysis: Sparsity Across Model Sizes

4.2.1 Model: [GPT2, GPT2-XL], Target Module: FFN, Dataset: CBT-CN

As shown in Figure 2, the accuracy of GPT2 remains close to its original level (around 84%) until the density ratio decreases to 70%. In contrast, GPT2-XL maintains its original accuracy (around 91%) even when the density ratio drops to 30%.

Furthermore, to achieve an accuracy of 80%, GPT2 requires approximately 70% of its FFN weights to be active, whereas GPT2-XL needs only 30%. However, both models retain a similar number of effective parameters. Using the hidden dimensions provided in Table 1, we can calculate the number of intermediate neurons that are activated:

- GPT-2 (Small): $3072 \times 0.7 \approx 2150$
- GPT-2 XL (Large): $6400 \times 0.3 = 1920$

These calculations reveal that the remaining capacities are comparable in magnitude. This suggests the CBT-CN dataset has a fixed intrinsic dimension, requiring approximately 2000 active dimensions to solve effectively. The larger model contains significant redundancy, allowing it to be aggressively reduced to a density of 0.3 while preserving this capacity. In contrast, the smaller model lacks spare neurons and requires a much higher density of 0.7 to maintain the same functional power.

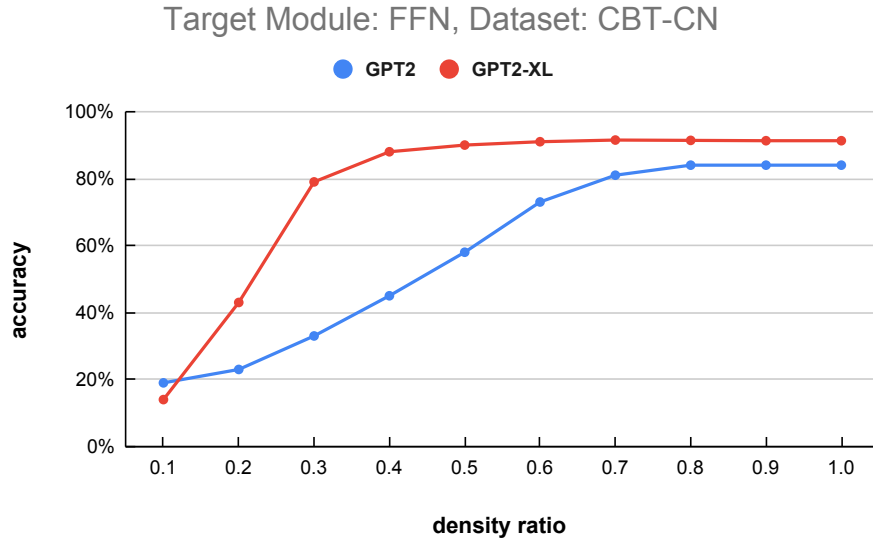


Figure 2

Table 1: Dimensions of weight matrices of GPT-2

	GPT2	GPT2-XL
input dimension (d_{in})	768	1600
hidden dimension (h)	3072	6400
output dimension (d_{out})	768	1600

4.2.2 Model: [GPT2, GPT2-XL], Target Module: FFN, Dataset: WikiText2

As shown in Figure 3, the perplexity of GPT2-XL remains stable and low (≤ 500) until the density ratio decreases to 20%. In contrast, GPT2 begins to degrade significantly once the density drops below 40%. This confirms that the larger model has higher redundancy, allowing it to withstand aggressive sparsity without losing its predictive capability.

Furthermore, to maintain a stable low perplexity, GPT2 requires approximately 40% of its FFN weights to be active, whereas GPT2-XL needs only 20%. Despite this difference in density, both models retain a similar number of effective parameters at their respective breaking points. Using the hidden dimensions provided in Table 1, we can estimate the critical capacity required for this dataset:

- GPT-2 (Small): $3072 \times 0.4 \approx 1228$
- GPT-2 XL (Large): $6400 \times 0.2 = 1280$

These calculations reveal that the effective capacities are remarkably close. This suggests that the WikiText2 dataset has a fixed intrinsic dimension, requiring approximately 1200 active dimensions for the model to model the language distribution effectively. The larger model can sustain this capacity even when activation is restricted to a 0.2 density, whereas the smaller model must maintain a density of 0.4 to preserve the same functional power.

4.2.3 Key Findings

The results across both benchmarks demonstrate a clear relationship between model scale and contextual sparsity. While the larger GPT2-XL model can maintain performance at significantly lower density ratios compared to the smaller GPT2, the *absolute* computational capacity required to solve a specific task remains constant. As calculated above, both models effectively "break" at a similar threshold of active neurons for a given dataset (approximately 2000 for CBT-CN and 1200 for WikiText2), regardless of their total size. This implies that task complexity imposes a fixed intrinsic dimensionality. Consequently, as model size increases, the additional parameters primarily contribute to redundancy. This allows larger models to route information through a smaller *percentage* of their total parameters while preserving the necessary active sub-network, making them naturally more amenable to acceleration via contextual sparsity.

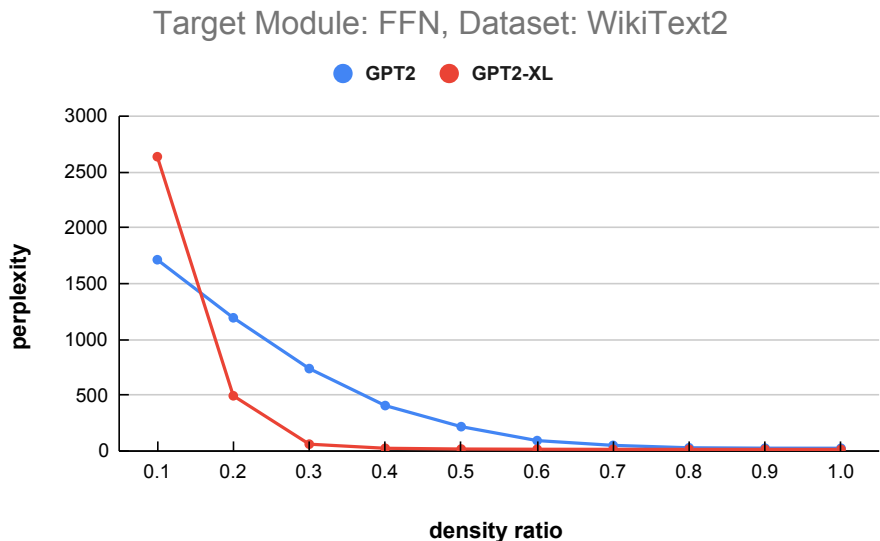


Figure 3

4.3 Component Analysis: Attention versus Feed-Forward Sparsity

4.3.1 Model: GPT2, Target Module: [Attention, FFN], Dataset: CBT-CN

Figure 4 compares the sparsity sensitivity of Attention and FFN modules within the GPT2 model on the CBT-CN dataset. The FFN module exhibits high resilience; even when pruned to a density of 0.7, it retains an accuracy of approximately 80%. This suggests that FFN layers contain significant redundancy. Conversely, the blue line shows that the Attention mechanism is highly sensitive to sparsity. At the same density level (0.7), the accuracy collapses to roughly 40%. This disparity indicates that while FFN parameters can be aggressively sparsified, Attention weights are critical for the model’s reasoning capabilities and must be preserved to maintain performance.

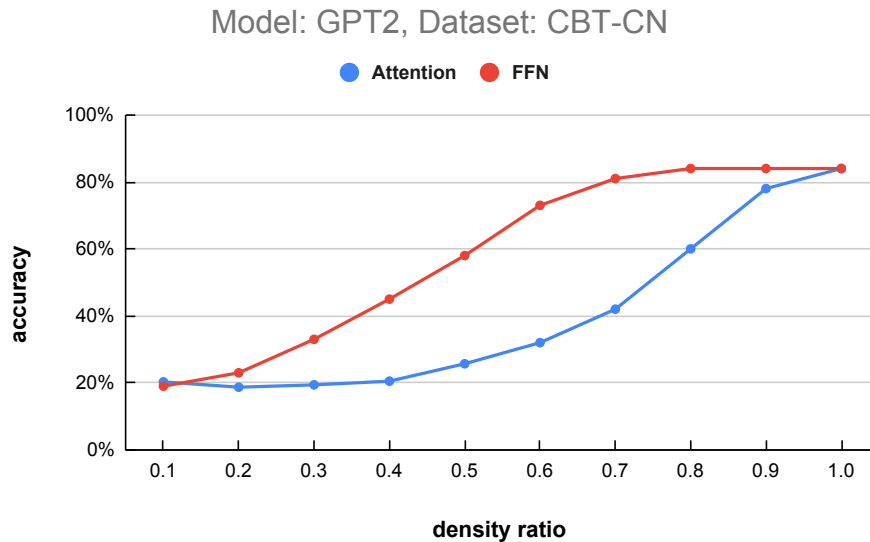


Figure 4

4.3.2 Model: GPT2-XL, Target Module: [Attention, FFN], Dataset: CBT-CN

As shown in Figure 5, the Attention layer and FFN layer maintain a high accuracy of over 80% even when pruned down to a density of 0.3. In fact, at extreme sparsity levels (0.2 density), the Attention module actually outperforms the FFN module, retaining approximately 65% accuracy compared to FFN’s 42%.

This finding suggests that redundancy scales with model size. Unlike small models where Attention heads are critical and non-redundant, large models have spare Attention capacity, allowing them to withstand aggressive compression across all components, not just in FFN layers.

Comparing with the smaller model (GPT2) shown in Figure 4, we observe a critical difference in how model size affects the robustness of the Attention mechanism. In the GPT2 model, the Attention module is extremely fragile, with performance collapsing immediately once density drops below 0.9. In sharp contrast, the Attention layer in the larger GPT2-XL exhibits remarkable resilience, maintaining an accuracy of over 80% even when sparsified to a low density of 0.3.

This comparison reveals a positive correlation between model size and the sparsity tolerance of Attention mechanisms. While Attention heads act as fragile "load-bearing" components in smaller architectures, scaling up the model introduces massive redundancy that neutralizes this vulnerability. Remarkably, in the larger GPT2-XL, the Attention module not only withstands aggressive compression but effectively surpasses the FFN module's robustness at extreme sparsity levels.

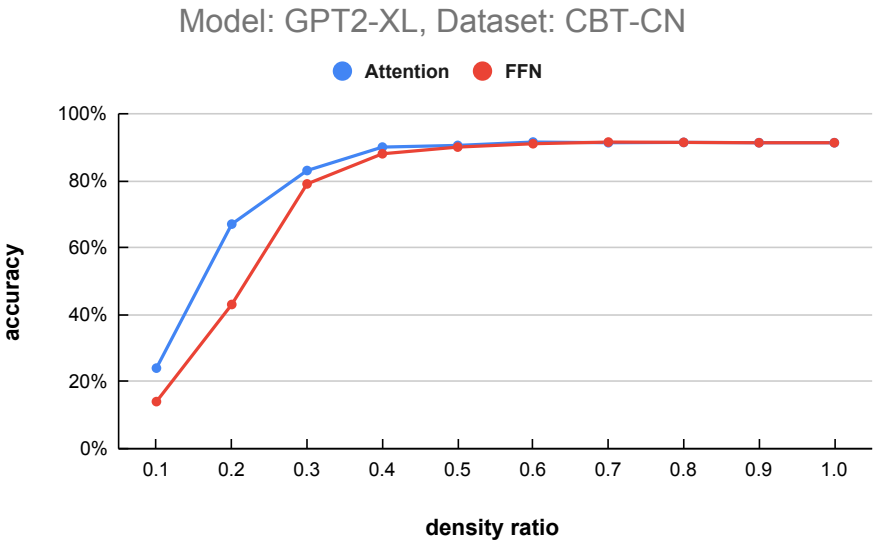


Figure 5

4.3.3 Model: Llama3-8B, Target Module: [Attention, FFN], Dataset: MGSM

Figure 6 highlights a significant disparity in pruning sensitivity between the Attention and FFN layers within the Llama3-8B model. The results demonstrate the remarkable resilience of FFN layers. Even when sparsified to a density of just 30%, the accuracy remains stable at approximately 55%. This suggests that FFN layers possess high redundancy, acting as robust memory stores that can withstand aggressive compression.

In sharp contrast, the Attention layers prove to be highly fragile. Performance collapses precipitously as density decreases, with accuracy dropping to 55% as soon as density falls below 0.8. This sensitivity arises because Attention mechanisms are critical for information routing and contextual focus, which are essential for distinguishing data categories. Consequently, while FFN weights can be aggressively sparsified, Attention weights are bearing components that must be preserved to maintain the model’s functionality.

4.3.4 Key Findings

The comparative analysis across different architectures reveals a consistent dichotomy between the sparsity tolerance of FFN and Attention modules. Across all experiments, FFN layers demonstrated high redundancy, functioning effectively as robust memory stores even at low density ratios (30–40%) without significant performance loss. Conversely, Attention modules frequently emerged as the performance bottleneck. While extreme model scaling on simpler

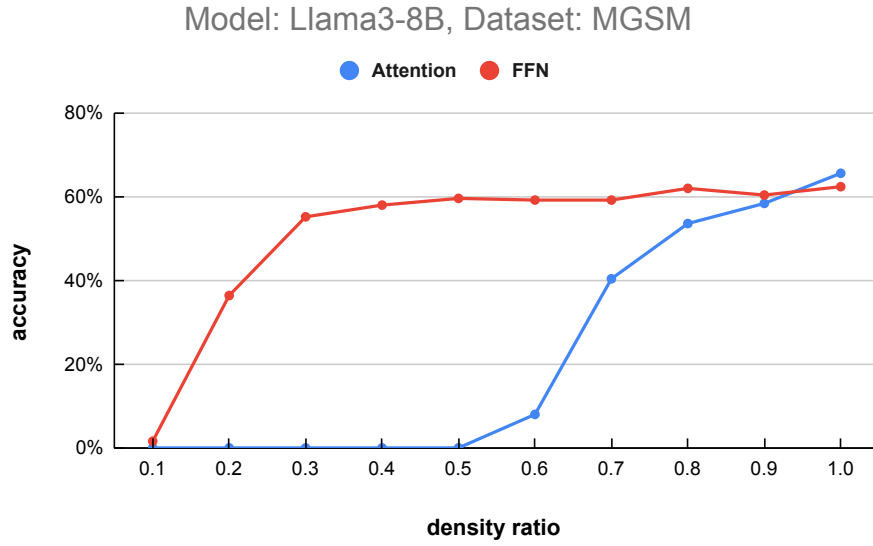


Figure 6

datasets (GPT2-XL on CBT-CN) hinted that scale can introduce redundancy into Attention heads, the sharp performance collapse observed in Llama3-8B on the MGSM benchmark indicates that Attention mechanisms remain highly sensitive, particularly for complex reasoning tasks. Therefore, we conclude that Contextual Sparsity strategies are most safely applied to FFN modules, which consistently offer substantial acceleration potential with minimal risk to model accuracy, whereas Attention pruning requires distinct, more conservative handling.

5 Conclusion

This study evaluates the efficacy of contextual sparsity in Transformer models, leading to three key findings. **First, sparsity is governed by task complexity.** Harder tasks requiring long-range dependency tracking necessitate a higher density of active parameters, confirming that complex problems engage a larger portion of the model’s capacity. **Second, scaling introduces redundancy.** We observed that datasets impose a fixed intrinsic dimension, implying that the number of neurons required to solve a task remains constant. Larger models possess vast spare capacity above this threshold, allowing them to be aggressively sparsified while preserving the same effective power as smaller, denser models. **Third, Attention and FFN modules exhibit distinct sensitivities.** Feed-Forward Networks (FFNs) consistently act as robust, sparse memory stores safe for aggressive sparsity. In contrast, Attention mechanisms are generally fragile bottlenecks, though extreme model scaling can eventually introduce sufficient redundancy to stabilize them.

Collectively, these results confirm that contextual sparsity is a dynamic property of Large Language Models. By exploiting the inherent redundancy in FFNs and the massive capacity of scaled architectures, inference can be significantly accelerated without compromising accuracy.

References

- [1] Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, and Beidi Chen. Deja vu: Contextual sparsity for efficient LLMs at inference time. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 22137–22176. PMLR, 23–29 Jul 2023.